

August 21, 2026

Continuous Prompt Evaluation: How We Use LLM Judges and Live Signals to Improve Kiro Agent Quality



Myeongsoo Kim
Applied Science



Patrick Chapman
Applied Science



Sai Srinivas Somarouthu
Developer



Jay Agrawal
Developer

By



Murali Krishna Ramanathan
Applied Science

The problem: system prompt behavior is hard to predict

Prompt behavior is difficult to validate exhaustively. A system prompt operates across combinations of models, tools, codebases, tasks, and users that no test suite can fully cover. An instruction that looks reasonable in isolation can cause unintended behavior in cases the prompt author did not anticipate, and a change that improves one scenario can regress another.

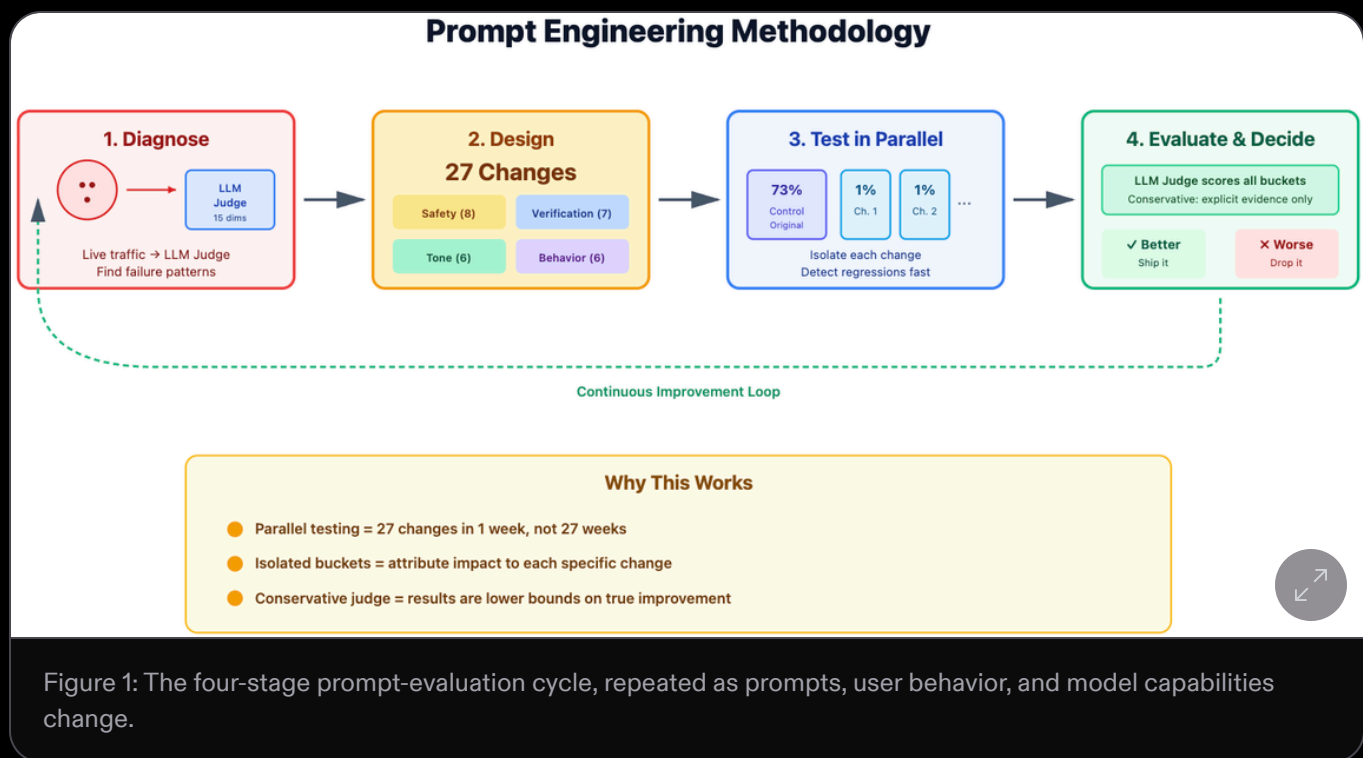
Prompt staleness is one instance of this broader problem. An instruction may already cause rare or hard-to-observe issues, then become more visibly harmful when a newer model follows it more strictly. New tools, workflows, and user patterns can expose the same latent defects.

Benchmarks remain an important quality signal, but they cannot capture every behavior that appears in everyday development work. We needed a process that detects issues on real workflows, categorizes recurring patterns, traces them to prompt instructions, and validates fixes as models and usage evolve.

The system at a glance

This article focuses on evaluating human-authored system prompt and configuration changes. It does not cover automated trajectory mining, change generation, or broader harness self-improvement. The workflow pairs benchmark results with an LLM-based analysis of thousands of Kiro conversations with our internal developers to identify opportunities in task completion, verification, and tool use. It has four stages:

- **Diagnose** high-frequency complaint categories and trace them to prompt instructions
- **Design** targeted prompt changes for isolated evaluation
- **Test** the changes in controlled cohorts on internal traffic
- **Evaluate** each cohort with the LLM judge for changes in dissatisfaction and behavioral quality issues



The four-stage prompt-evaluation cycle

The diagram above summarizes the same four stages described below. We repeat this cycle as prompts, user behavior, and model capabilities change:

1. Diagnose. We run the judge on current internal traffic to identify and categorize recurring complaint patterns, then trace each high-frequency cluster to a contributing prompt instruction or missing instruction. For example, recurring "incomplete task" complaints pointed to guidance that encouraged the agent to explain its plan rather than execute it.

2. Design. We write targeted candidate changes for the diagnosed issue. Candidates commonly address safety, verification, tone, or behavior, and each candidate states the intended behavior

and regression risk before testing.

3. Test. We compare candidates with the control in isolated cohorts and, when useful, in a combined cohort that can reveal interactions. The experiment design and sample size determine whether a result is only directional or supports a broader conclusion.

[← Back to all posts](#)

4. Evaluate. We apply the same LLM-judge rubric to each cohort, compare dissatisfaction and behavioral-quality rates, and use the available evidence to ship, revise, or reject each candidate.



We built an evaluation framework that scores internal conversations across 15 behavioral dimensions, including:

- *Task completeness*
- *Claim accuracy* (did the agent verify before asserting?)
- *Code style adherence*
- *Recognition of repeated unsuccessful approaches*
- *Destructive-action flagging*
- *Tool use appropriateness*
- *Verification behavior* (running tests, checking compilation)

The judge requires explicit evidence from the conversation: corrections, complaints, abandoned tasks, or confirmations. Ambiguous conversations do not count against either variant, keeping comparisons consistent and conservative to avoid excessive false-positive judgments.

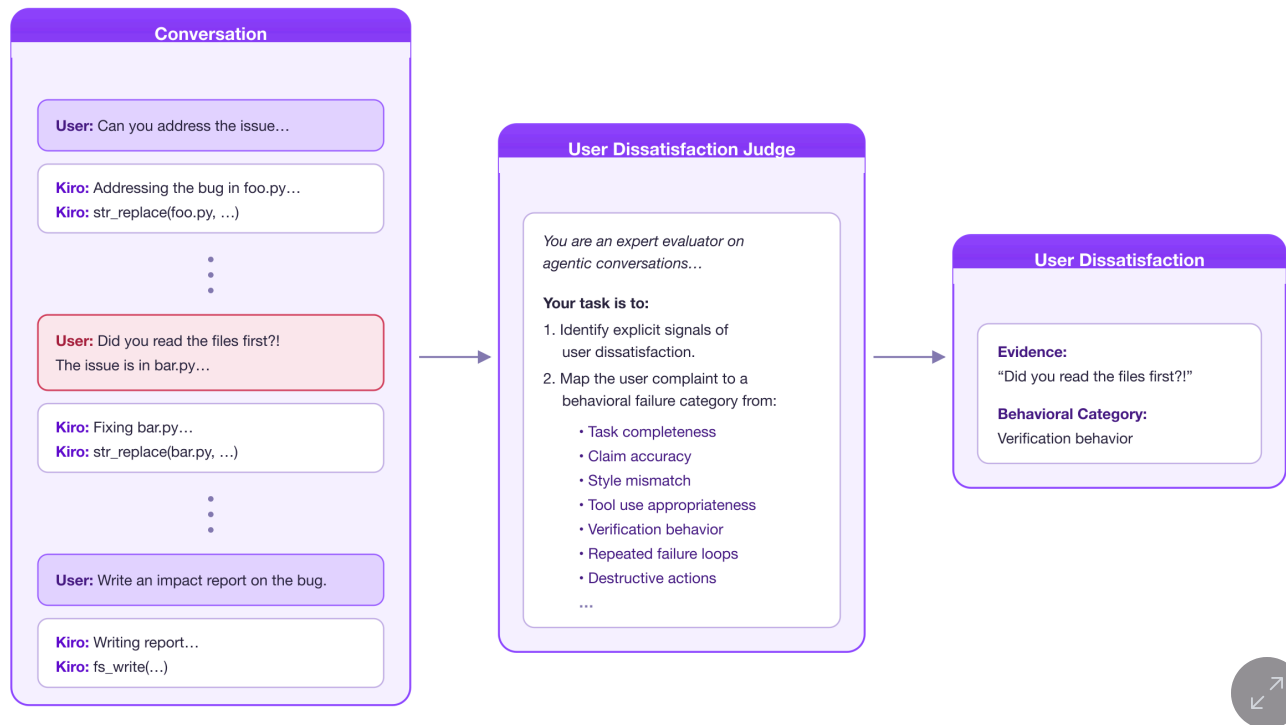


Figure 2: How the LLM judge reads a conversation—chat exchanges on the left, dissatisfaction criteria in the middle, and categorized behavioral-quality output on the right.

The judge prompts represented in the figure are minimal illustrative examples and are not what is actually used during evaluation.

The judge measures two primary signals:

- **Explicit dissatisfaction:** Did the user explicitly express frustration, abandon the conversation, or redo the agent's work? This is inferred only from explicit feedback in the conversation, not from implied tool results or survey scores.
- **Behavioral quality issues:** Did the agent miss one of the 15 quality standards? Examples include making claims without first reading the code, stopping before completing the task, or disregarding the project's existing patterns.

Both metrics use the same rubric across control and treatment groups, supporting consistent comparisons between configurations within the same evaluation design.

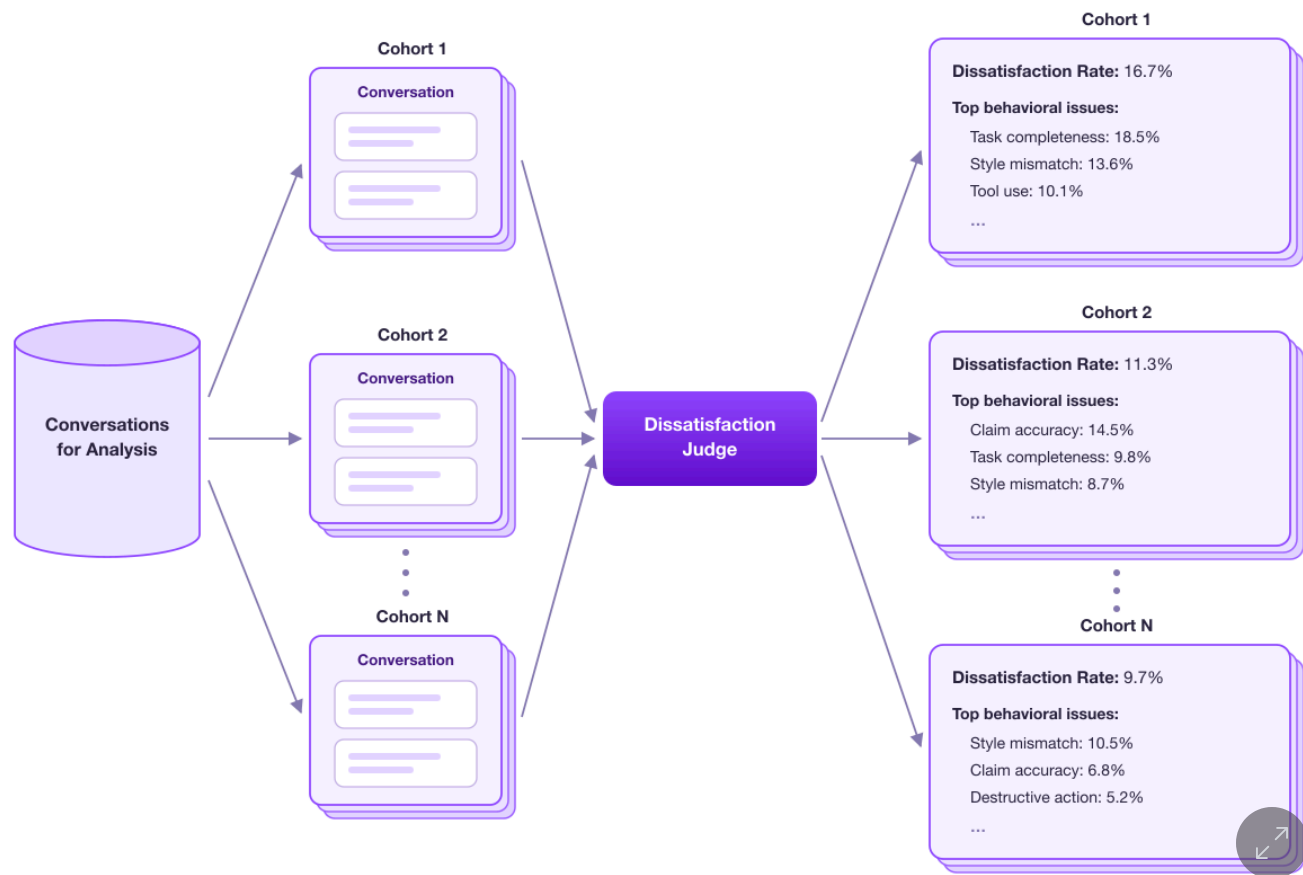


Figure 3: Conversations flow from storage through per-cohort buckets into the dissatisfaction judge, which outputs dissatisfaction rates and behavioral-issue counts per cohort.

A/B experiment infrastructure

Before shipping a prompt change, we compare configurations in separate cohorts on internal traffic. For these experiments, the service assigns eligible internal developers to stable user buckets using a deterministic hash of user identity, independent of observed conversation outcomes. Each bucket maps to one prompt variant for a given experiment.

The judge pipeline analyzes conversations by their recorded experiment allocation and applies the same rubric to every cohort. Stable assignment controls cohort selection within the prompt experiment, but it does not eliminate every source of bias or interaction in live traffic. Exact allocation and sample size vary by experiment, so the strength of a conclusion depends on the evidence available for that comparison. Small isolated candidate cohorts remain directional diagnostics rather than precise estimates of production impact.

In one accelerated internal deployment, we screened 27 candidate prompt changes across four categories: safety, verification, tone, and behavior. We compared a stable control with isolated candidate cohorts and a cohort that combined the candidates. The small isolated cohorts were directional diagnostics for localizing possible regressions, not individually powered estimates of

impact. Candidates that showed degradation were revised or removed, while combined and larger follow-up comparisons supported broader conclusions.

Because the judge pipeline scores conversation outcomes, it can also compare user-visible configuration choices. We have used it to evaluate default reasoning effort levels, where common tasks such as code modification and debugging showed gains from higher effort levels. For more information on configuring reasoning effort levels, see the [reasoning effort documentation](#).

Results from recent cycles

The figures below are observed deltas from internal experiment-level comparisons. They describe these evaluation samples and should not be read as universal production-effect estimates.

Initial model-and-prompt experiments

Kiro CLI (first evaluated model-and-prompt configuration):

- Explicit dissatisfaction signals: **down 5%**
- Behavioral quality issues: **down 32%**
- Task completeness issues: **down 10.6%**

Kiro IDE (first evaluated model-and-prompt configuration):

- Behavioral quality issues: **down 20%**
- Incomplete task delivery: **down 21%**
- Repeated unsuccessful approaches: **down 36%**
- Style mismatches: **down 54%**

Re-validation on a newer model

Re-validation answers a different question from the initial experiments. Those experiments measured the effect of prompt changes on the first evaluated model-and-prompt configuration. Re-validation tests whether the same changes still help after a model upgrade.

The newer model-and-prompt baseline already had fewer behavioral quality issues under the same rubric, so there was less headroom for a prompt to add; the gains here are therefore smaller than the initial cycle's by design, not a regression. Even against that harder baseline, the same prompt changes still moved both signals the right way: they reduced behavioral quality

issues by a further 4% and improved explicit dissatisfaction by 2.6 percentage points. We read this pass as a directional confirmation that the changes still help after a model upgrade, not as a precise production estimate.

The newer model also followed some instructions more literally, especially at lower effort levels, so guidance tuned for the previous model did not transfer cleanly in every case. We therefore treat each model upgrade as a new model-and-prompt configuration: replay known complaint cases, use previously successful cases as regression checks, measure behavioral quality, dissatisfaction, regressions, and efficiency, then retune before rollout.

Takeaways for teams evaluating their own agentic systems

Live usage evaluation complements benchmarks. Standard benchmarks remain useful. Internal feedback reveals opportunities for system prompt improvement from real user sessions that often contain dimensions that standard benchmarks do not capture together, such as user-specific workspace context, interaction with real production services, conversations resumed across separate working sessions, and other real-world noise.

Use isolated cohorts for directional screening. Small candidate cohorts can help localize regressions, while combined and adequately powered follow-up comparisons are needed before drawing broader conclusions.

Prompt effects are model-dependent. The same changes reduced behavioral quality issues by 32% on one model version and by 4% on another. Every model upgrade needs a re-validation pass.

Retire guidance that is no longer needed. Some of the largest gains come from removing outdated constraints, including line limits and deprecated tool workarounds. Simpler prompts are easier to keep aligned with model behavior.

What this means for Kiro users

These changes make Kiro more likely to complete the full task, change course when an approach is not working, and follow the project's existing code style. They also encourage Kiro to verify that changes compile and tests pass before moving on, and to ask before risky actions while proceeding directly with safe work. We repeat this prompt evaluation as prompts and models change, validating updates before deployment.

Share



[{ PREVIOUS }](#)

How We Learned to Trust an AI Agent to
Triage Production Incidents

KIRO

PRODUCT

[About Kiro](#)[IDE](#)[CLI](#)[Web](#)[Mobile](#)[Crew](#)[Pricing](#)[Downloads](#)

COMMUNITY

[Overview](#)[Ambassadors](#)[Discord](#)[Events](#)[Powers](#)[Shop](#)[Showcase](#)

SOCIAL

FOR

[Enterprise](#)[Startups](#)[Students](#)

RESOURCES

[Docs](#)[Blog](#)[Changelog](#)[FAQs](#)[Report a bug](#)[Suggest an idea](#)[Billing support](#)



[Site Terms](#) [License](#) [Responsible AI Policy](#) [Legal](#) [Privacy Policy](#) [Cookie Preferences](#)



 ENGLISH 